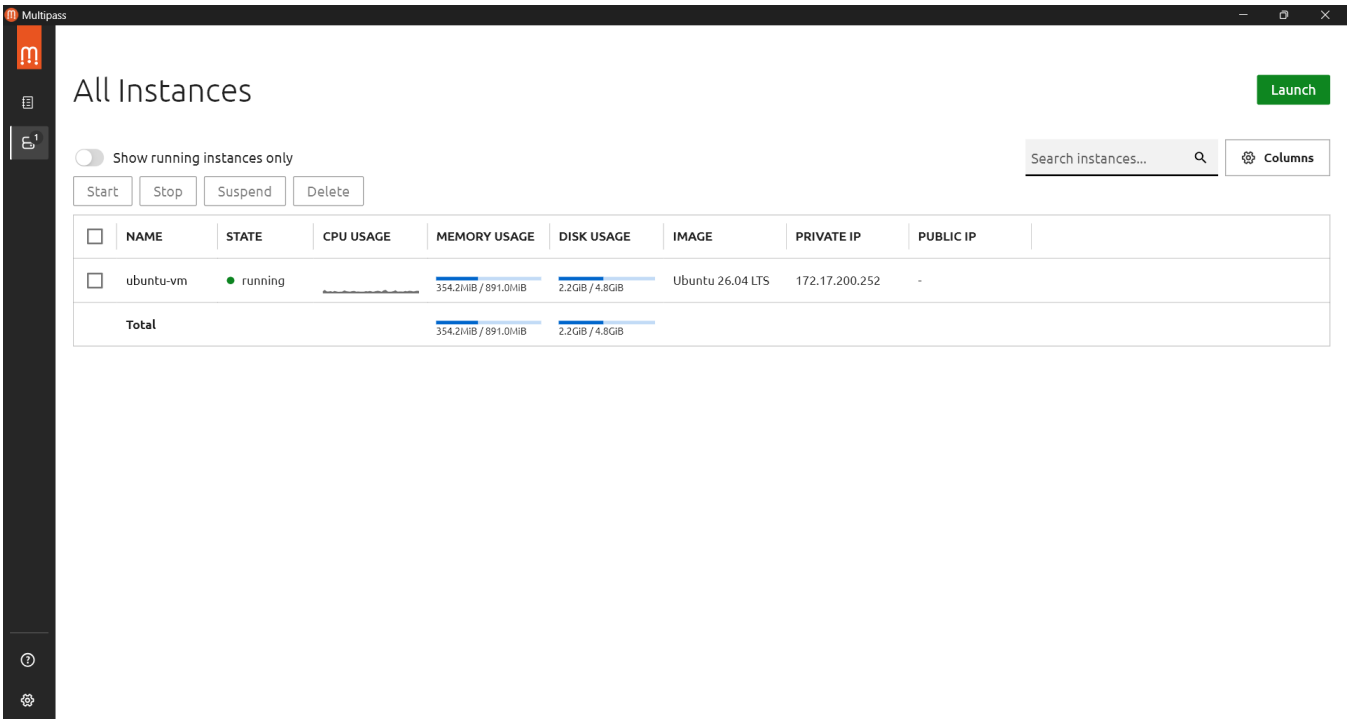
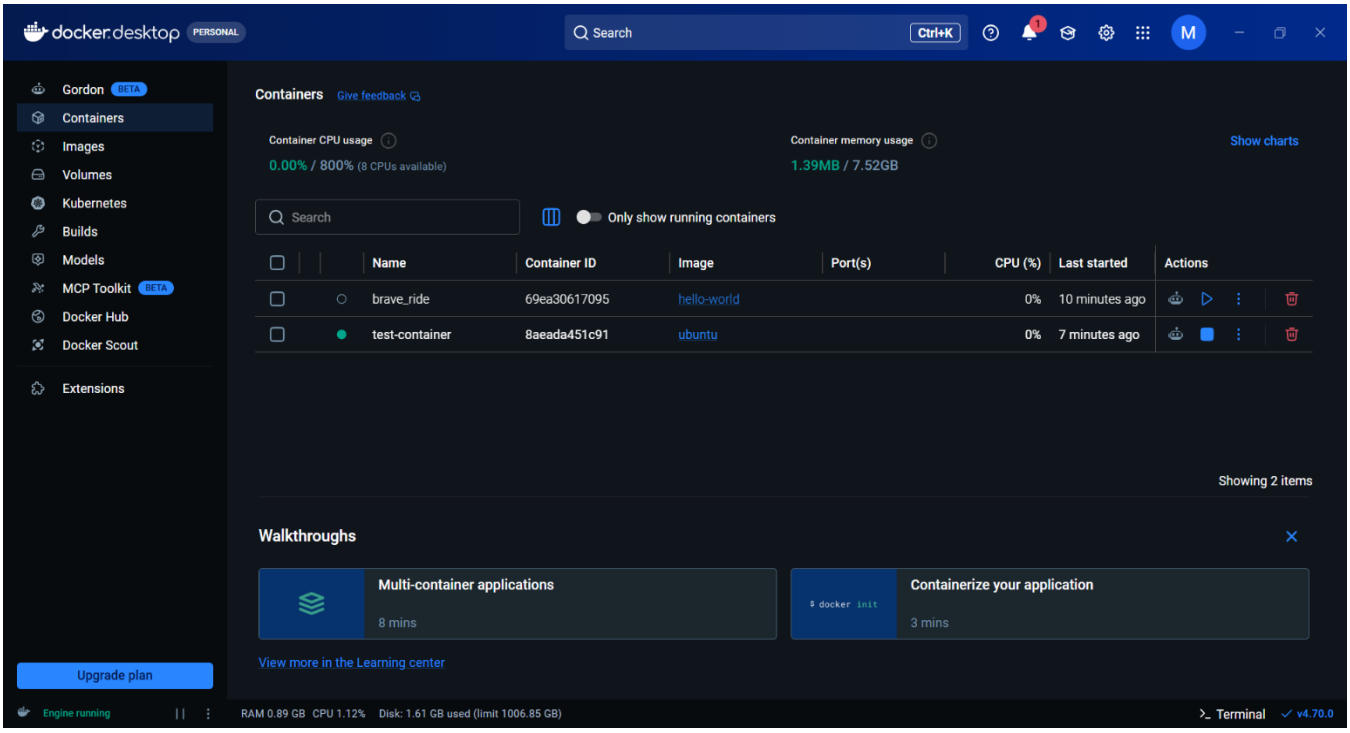
Name: Menna Hesham Ali - Group:SE2

# Lab1 - Exploring Cloud Virtualization and Data Center Architecture

- **VM:**




- **Docker:**



- **Observations:** 1. Memory Usage

**VM:**

```
C:\Users\Menna>multipass shell ubuntu-vm
Welcome to Ubuntu 26.04 LTS (GNU/Linux 7.0.0-14-generic x86_64)

 * Documentation:  https://docs.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

 System information as of Fri Apr 24 19:39:43 EEST 2026

  System load:  0.1               Processes:             112
  Usage of /:   56.7% of 3.70GB   Users logged in:       0
  Memory usage: 26%               IPv4 address for eth0: 172.17.200.252
  Swap usage:   0%


Expanded Security Maintenance for Applications is not enabled.

2 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status


ubuntu@ubuntu-vm:~$ free -h
               total        used        free      shared  buff/cache   available
Mem:           891Mi       345Mi       208Mi       3.9Mi       475Mi       545Mi
Swap:             0B          0B          0B
```

The VM has fixed allocated memory (~891 MB)

**Container:**

```
C:\Users\Menna>docker run -it --name test-container ubuntu /bin/bash
Unable to find image 'ubuntu:latest' locally
latest: Pulling from library/ubuntu
b40150c1c271: Pull complete
92842f25412d: Download complete
Digest: sha256:c4a8d5503dfb2a3eb8ab5f807da5bc69a85730fb49b5cfca2330194ebcc41c7b
Status: Downloaded newer image for ubuntu:latest
root@8aeada451c91:/# free -h
               total        used        free      shared  buff/cache   available
Mem:           7.7Gi       850Mi       6.1Gi       3.2Mi       995Mi       6.9Gi
Swap:          2.0Gi          0B       2.0Gi
```

The container sees the host's full memory (7.7 GB)

- **Conclusion:**

VM = isolated, fixed resources
Container = shares host resources

2. Number of Processes

**VM:**

```
ubuntu@ubuntu-vm:~$ ps aux --sort=-%mem | head
USER         PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root        1627  0.0  3.5 126456 32732 ?        Ssl  19:33   0:00 /usr/bin/python3 /usr/share/unattended-upgrades/unattended-upgrade-shutdown --wait-for-si
gnal
root        1525  0.0  3.2  47088 29796 ?        Ss   19:33   0:00 /usr/bin/python3 /usr/bin/networkd-dispatcher --run-startup-triggers
root           1  0.5  1.7  25188 16348 ?        Ss   19:33   0:03 /usr/lib/systemd/systemd --switched-root --system --deserialize=44
root        1536  0.0  1.6 543436 15000 ?        Ssl  19:33   0:00 /usr/libexec/udisks2/udisksd
systemd+     958  0.0  1.6  22744 14872 ?        Ss   19:33   0:00 /usr/lib/systemd/systemd-resolved
root         963  0.0  1.4  39356 13552 ?        Ss   19:33   0:00 /usr/lib/systemd/systemd-udevd
root        1666  0.0  1.4 391608 13416 ?        Ssl  19:33   0:00 /usr/sbin/ModemManager
ubuntu      1796  0.1  1.4  22184 12992 ?        Ss   19:33   0:00 /usr/lib/systemd/systemd --user
systemd+    1362  0.0  1.3  21464 12428 ?        Ss   19:33   0:00 /usr/lib/systemd/systemd-networkd
```

100+ processes, includes:
- systemd
- udisksd
- ModemManager
- networking services

**Docker:**

```
root@8aeada451c91:/# ps aux --sort=-%mem | head
USER       PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root         1  0.0  0.0   4588  4044 pts/0    Ss   16:40   0:00 /bin/bash
root        10  0.0  0.0   7888  4020 pts/0    R+   16:43   0:00 ps aux --sort=-%mem
root        11  0.0  0.0   2708  1116 pts/0    S+   16:43   0:00 head
```

Only 2–3 processes total

- **Conclusion:**

VM = full operating system running
Container = just one application + minimal processes

3. Disk Usage

**VM**

```
ubuntu@ubuntu-vm:~$ df -h
Filesystem      Size  Used Avail Use% Mounted on
tmpfs           179M  948K  178M   1% /run
/dev/sda1       3.8G  2.1G  1.6G  57% /
tmpfs           446M     0  446M   0% /dev/shm
efivarfs        128M   32K  128M   1% /sys/firmware/efi/efivars
tmpfs           446M     0  446M   0% /tmp
none            1.0M     0  1.0M   0% /run/credentials/systemd-journald.service
none            1.0M     0  1.0M   0% /run/credentials/systemd-resolved.service
/dev/sda13      989M  106M  816M  12% /boot
/dev/sda15      105M  6.3M   99M   7% /boot/efi
none            1.0M     0  1.0M   0% /run/credentials/systemd-networkd.service
tmpfs            90M  8.0K   90M   1% /run/user/1000
none            1.0M     0  1.0M   0% /run/credentials/getty@tty1.service
none            1.0M     0  1.0M   0% /run/credentials/serial-getty@ttyS0.service
```

/dev/sda1 → 3.8G total

**Docker**

```
root@8aeada451c91:/# df -h
Filesystem      Size  Used Avail Use% Mounted on
overlay        1007G  1.7G  955G   1% /
tmpfs            64M     0   64M   0% /dev
tmpfs           3.9G     0  3.9G   0% /sys/fs/cgroup
shm              64M     0   64M   0% /dev/shm
/dev/sdd       1007G  1.7G  955G   1% /etc/hosts
tmpfs           3.9G     0  3.9G   0% /proc/acpi
tmpfs           3.9G     0  3.9G   0% /sys/firmware
root@8aeada451c91:/#
```

overlay → ~1000G (host disk)

- **Conclusion:**

VM has its **own virtual disk**
Container uses **host filesystem (overlay)**

4. Startup Behavior

- **VM** → took time (download + boot OS)
- **Container** → started instantly

**Q: Which architecture (VM or container) is better for microservices? Why?**

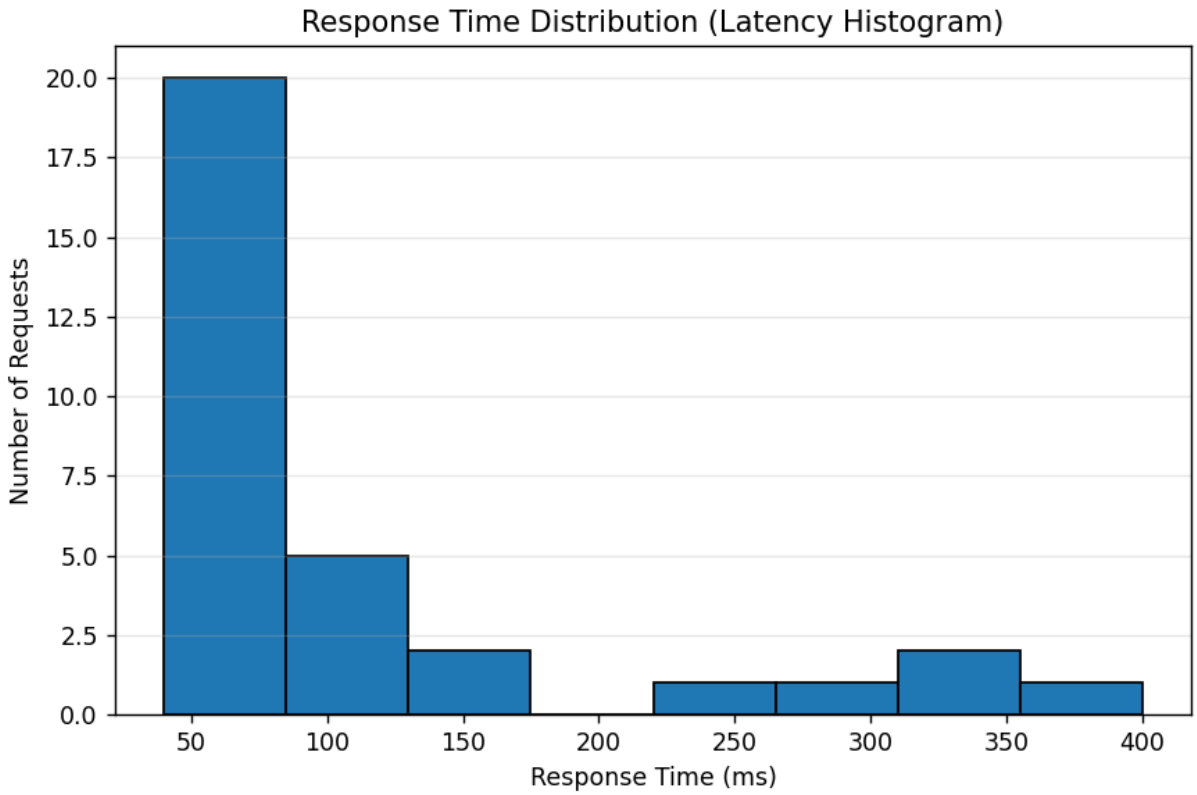
**Containers are better for microservices because:**

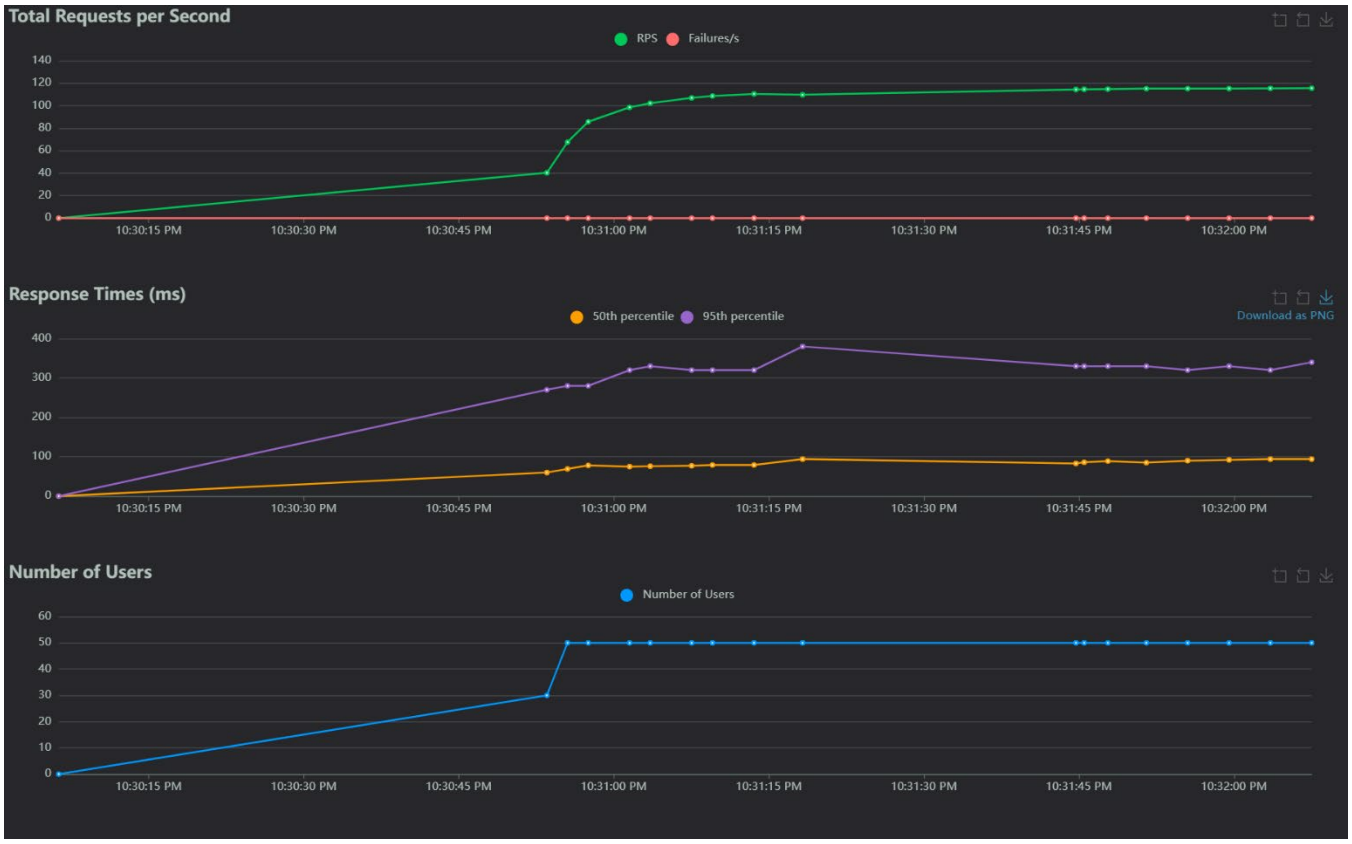
- Lightweight (few processes)
- Fast startup (milliseconds vs seconds)
- Efficient resource usage
- High density (many containers per machine)

**While VMs are better when:**

- You need strong isolation
- Running different OS types
- Security-critical workloads

**Latency Graph of web service:**






- The graph shows that system performance is not defined by average response time, but by the slowest requests. While most requests complete quickly, a small fraction experience high latency, forming the “tail” of the distribution.

- As concurrency increases, resource contention leads to more frequent and larger latency spikes. This highlights the importance of considering tail latency in the design of scalable cloud systems.